

1 (a)

Clock frequency of computer A = 2 GHz

Total cycles required = $10 \times 3 + 5 \times 6 + 4 \times 2$

Total cycles = $30 + 30 + 8 = 68$ cycles

Execution time = $68 / 2 = 34$ ns

Affected Time: 34 ns

Unaffected Time: $55 - 34 = 21$ ns

(b)

Execution time of B = Half of execution time of A = $34 / 2 = 17$ ns

Total cycle since same ISA = 68

Required clock frequency = $68 / 17 = 4$ GHz

(c)

Time for each instruction

R type = $10 \times 3 / 2 = 15$ ns I type = $5 \times 6 / 2 = 15$ ns J type = $4 \times 2 / 2 = 4$ ns

System time = 21 ns

If r type is improved 3 times, new time for r type = $15 / 3 = 5$ ns

New Response time if r type is improved = $T_{\text{improved}} = 5 + 15 + 4 + 21 = 45$ ns

$T_{\text{affected}} = 15$ ns, $T_{\text{unaffected}} = 55 - 15 = 40$ ns

$45 = 15 / IF + 40$

$IF = 15 / 5 = 3$

2

(a)

my_function:

addi \$sp, \$sp, -4 # allocate 4 bytes for sum

sw \$zero, 0(\$sp) # sum = 0

loop:

slt \$t0, \$a1, \$a0 # $t0 = (b < a)$

bne \$t0, \$zero, exit # if $(b < a)$ exit

address of arr[b]

sll \$t1, \$a1, 2 # $t1 = b * 4$

add \$t1, \$a2, \$t1 # $t1 = \&\text{arr}[b]$

```
lw $t2, 0($t1)    # t2 = arr[b]
```

You can also write 4* [register where you store b] in the offset place

```
# arr[b] * 16
sll $t3, $t2, 4    # t3 = arr[b] * 16
```

```
# (arr[b] * 16) % 32
andi $t4, $t3, 31  # t4 = % 32
```

```
# ~(...)
nor $t5, $t4, $zero # t5 = ~(t4)
```

```
# sum += t5
lw $t6, 0($sp)     # load sum
add $t6, $t6, $t5
sw $t6, 0($sp)     # store sum
```

```
# a++
addi $a0, $a0, 1
```

```
j loop # unconditional jump
```

exit:

```
lw $v0, 0($sp)     # move sum to return register
addi $sp, $sp, 4    # deallocate stack
jr $ra
```

2 (b)

1) bne \$s0, \$t8, IF (address 104)

op	rs	rt	immediate
5	16	24	4

2) sub \$t3, \$s1, \$s5 (address 108)

op	rs	rt	rd	shamt	funct
0	17	21	11	0	34

3) ori \$t2, \$t7, 42 (address 112)

op	rs	rt	immediate
13	15	10	42

4) lw \$t1, 20(\$t2) (address 116)

op	rs	rt	immediate
35	10	9	20

5) slt \$t1, \$s2, \$s5 (address 124)

op	rs	rt	rd	shamt	funct
0	18	21	9	0	42

6) j my_function (address 128)

op	target
2	25

2 (c)

- Total instruction size = **32 bits**
- Opcode = **6 bits**
- Registers in processor P = **256 registers**

So each register field needs: $\log_2(256) = 8$ bits

Size for address field is $32 - 6 - 8 - 8 = 10$

Maximum value for the address field is $2^{10} - 1 = 1023$

Maximum size of index = $1023 / 4 = 255$

But in question, index field is 301. We cant write it.

If we consider sign bits, maximum value for address field is $2^9 - 1 = 511$

Maximum amount thorough which we can do and operation is 511. But in question it says 1024, which we cannot write.